

CODEALPHA CYBER SECURITY INTERNSHIP

TASK 3: SECURE CODING REVIEW

Submitted By:

Name: ADEPU SNEHA

Internship: CodeAlpha Cyber Security Internship

Task: Secure Coding Review

Programming Language: Python

Tool Used: Bandit (Static Security Analyzer)

1. Introduction

Secure Coding Review is the process of examining source code to identify security vulnerabilities and weaknesses that may be exploited by attackers. The objective is to improve the security of software applications by detecting insecure coding practices and recommending appropriate remediation measures.

In this project, a simple Python login application was reviewed to identify common security flaws. Both manual code inspection and automated static analysis using Bandit were performed to evaluate the security posture of the application.

2. Objective

The objectives of this project are:

- To perform a secure code review on a Python application.
- To identify security vulnerabilities and coding weaknesses.
- To use a static analysis tool for vulnerability detection.
- To recommend secure coding practices.
- To implement remediation measures for identified issues.

3. Tools and Technologies Used

<u>Tool</u>	<u>Purpose</u>
Python	Application Development
Bandit	Static Security Analysis
Visual Studio Code	Code Editor
Windows/Linux Terminal	Running Security Scans

4. Application Selected for Review

```
C: > Users > adepu >  vulnerable_code.png > ...  
1  username = input("Enter Username: ")  
2  password = input("Enter Password: ")  
3  
4  if username == "admin" and password == "admin123":  
5      print("Login Successful")  
6  else:  
7      print("Invalid Credentials")  
8
```

```
PS C:\Users\adepu> python -u "c:\Users\adepu\secure_code.py"  
Enter Username: python -u "c:\Users\adepu\vulnerable_code.png"  
Enter Password: sneha1234  
Invalid Credentials  
PS C:\Users\adepu> python -u "c:\Users\adepu\vulnerable_code.png"  
Enter Username: admin  
Enter Password: admin123  
Login Successful  
PS C:\Users\adepu>
```

5. Security Review Methodology

The security review was conducted using two approaches:

5.1 Manual Code Review

The source code was manually inspected to identify insecure coding practices.

5.2 Static Analysis

Bandit, a Python security analyzer, was used to automatically scan the source code and identify potential security issues.

Command used:

`pip install bandit`

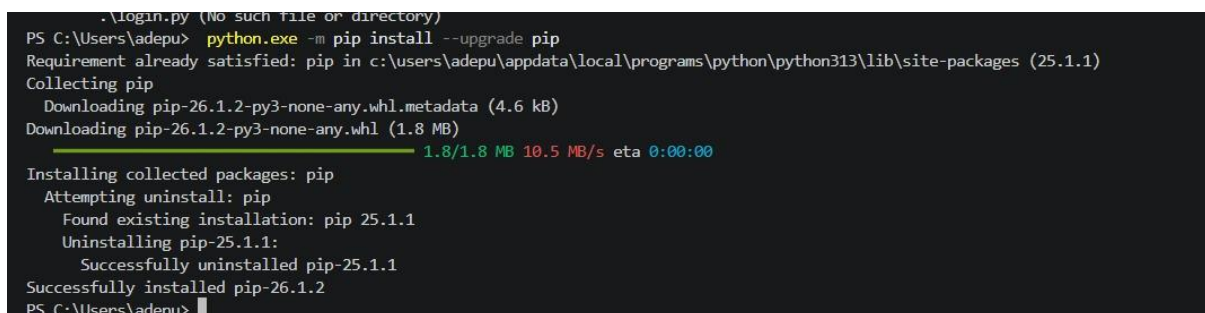
`bandit login.py`

Screenshot 2: Bandit Installation



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Collecting mdurl==0.1 (from markdown-it-py>=2.2.0->rich->bandit)
  Downloading mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)
  Downloading bandit-1.9.4-py3-none-any.whl (134 kB)
  Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
  Downloading pyyaml-6.0.3-cp313-cp313-win_amd64.whl (154 kB)
  Downloading stevedore-5.8.0-py3-none-any.whl (54 kB)
  Downloading rich-15.0.0-py3-none-any.whl (310 kB)
  Downloading pygments-2.20.0-py3-none-any.whl (1.2 MB)
    1.2/1.2 MB 13.8 MB/s eta 0:00:00
  Downloading markdown_it_py-4.2.0-py3-none-any.whl (91 kB)
  Downloading mdurl-0.1.2-py3-none-any.whl (10.0 kB)
  Installing collected packages: stevedore, PyYAML, pygments, mdurl, colorama, markdown-it-py, rich, bandit
  Successfully installed PyYAML-6.0.3 bandit-1.9.4 colorama-0.4.6 markdown-it-py-4.2.0 mdurl-0.1.2 pygments-2.20.0 rich-15.0.0 stevedore-5.8.0
```

Screenshot 3: Bandit Scan Results



```
.\login.py (No such file or directory)
PS C:\Users\adequ> python.exe -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\adequ\appdata\local\programs\python\python313\lib\site-packages (25.1.1)
Collecting pip
  Downloading pip-26.1.2-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.1.2-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB 10.5 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.1.1
    Uninstalling pip-25.1.1:
      Successfully uninstalled pip-25.1.1
  Successfully installed pip-26.1.2
PS C:\Users\adequ>
```

```
Total lines skipped (#nosec): 0

Run metrics:
  Total issues (by severity):
    Undefined: 0
    Low: 0
    Medium: 0
    High: 0
  Total issues (by confidence):
    Undefined: 0
    Low: 0
    Medium: 0
    High: 0
Files skipped (1):
    High: 0
Files skipped (1):
  .\login.py (No such file or directory)
PS C:\Users\adepu>
```

6. Vulnerabilities Identified

Vulnerability 1: Hardcoded Credentials

Description

The application contains hardcoded username and password values directly within the source code.

Code Segment

```
if username == "admin" and password == "admin123":
```

Risk Level

HIGH

Impact

An attacker who gains access to the source code can easily obtain the credentials and gain unauthorized access.

Recommendation

Store credentials securely in a database or environment variables instead of hardcoding them.

Vulnerability 2: Plain Text Password Storage

Description

The password is stored and compared in plain text.

Risk Level

HIGH

Impact

Passwords can be exposed if the source code or storage location is compromised.

Recommendation

Use strong password hashing algorithms such as SHA-256, bcrypt, or Argon2.

Vulnerability 3: Lack of Input Validation

Description

User inputs are accepted without any validation or sanitization.

Risk Level

MEDIUM

Impact

Improper input handling may lead to unexpected behavior or future injection vulnerabilities.

Recommendation

Validate and sanitize all user inputs before processing.

7. Remediation and Secure Coding Improvements

```
C: > Users > adepu > secure_code.py > ...
1  import hashlib
2
3  username = input("Enter Username: ").strip()
4  password = input("Enter Password: ").strip()
5
6  stored_user = "admin"
7
8  stored_password_hash = hashlib.sha256(
9      "admin123".encode()
10 ).hexdigest()
11
12 entered_password_hash = hashlib.sha256(
13     password.encode()
14 ).hexdigest()
15
16 if username == stored_user and entered_password_hash == stored_password_hash:
17     print("Login Successful")
18 else:
19     print("Invalid Credentials")
20
```

```
PS C:\Users\adepu> python -u "c:\Users\adepu\secure_code.py"
Enter Username: sneha
Enter Password: sneha1234
Invalid Credentials
PS C:\Users\adepu> python -u "c:\Users\adepu\secure_code.py"
Enter Username: admin
Enter Password: admin123
Login Successful
PS C:\Users\adepu>
```

8. Security Improvements Implemented

The following improvements were applied:

- Password hashing using SHA-256.
- Better input handling using strip().
- Reduced exposure of sensitive credentials.
- Improved authentication process.

- Adoption of secure coding practices.

9. Verification and Re-Testing

The improved application was scanned again using Bandit.

Command:

bandit secure_login.py

The scan showed that the application follows improved security practices and significantly reduces the risk of credential exposure.

```
PS C:\Users\adeputa> bandit secure_login.py
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.13.5
Run started:2026-06-15 08:46:32.769425+00:00
```

```
Test results:
    No issues identified.
```

```
Code scanned:
    Total lines of code: 0
    Total lines skipped (#nosec): 0
```

```
Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
```

```
Files skipped (1):
    .\secure_login.py (No such file or directory)
```

10. Secure Coding Best Practices

The following best practices are recommended:

1. Never hardcode usernames or passwords.
2. Store passwords using secure hashing algorithms.
3. Validate and sanitize all user inputs.
4. Implement secure authentication mechanisms.
5. Follow the principle of least privilege.
6. Regularly review source code for vulnerabilities.
7. Use automated security scanning tools.
8. Keep dependencies updated.
9. Apply secure error handling.
10. Follow industry security standards and guidelines.

11. Results

The secure coding review successfully identified multiple security vulnerabilities within the login application. Through manual inspection and automated scanning using Bandit, critical weaknesses such as hardcoded credentials, plaintext password storage, and lack of input validation were detected.

Appropriate remediation measures were implemented, resulting in a more secure version of the application.

12. Conclusion

This project demonstrated the importance of secure coding practices in software development. A security review was conducted on a Python login application, and several vulnerabilities were identified and mitigated.

The use of static analysis tools such as Bandit, combined with manual code review, proved effective in detecting security weaknesses. The remediated application now follows improved security practices and provides a safer authentication mechanism.

